

JavaCrush

Marco Galli, Gianluca Marsella, Sara Mazzoli, Anna Pollini

29 giugno 2026

Indice

1	Analisi	4
1.1	Descrizione e requisiti	4
1.2	Modello del Dominio	5
2	Design	7
2.1	Architettura	7
2.2	Design Dettagliato	9
2.2.1	Marco Galli	9
2.2.2	Gianluca Marsella	14
2.2.3	Sara Mazzoli	21
2.2.4	Anna Pollini	27
3	Sviluppo	32
3.1	Testing automatizzato	32
3.1.1	Marco Galli	32
3.1.2	Gianluca Marsella	33
3.1.3	Sara Mazzoli	35
3.1.4	Anna Pollini	36
3.2	Note di sviluppo	37
3.2.1	Marco Galli	37
3.2.2	Gianluca Marsella	38
3.2.3	Sara Mazzoli	39
3.2.4	Anna Pollini	41
4	Commenti finali	43
4.1	Autovalutazione e lavori futuri	43
4.1.1	Marco Galli	43
4.1.2	Gianluca Marsella	44
4.1.3	Sara Mazzoli	45
4.1.4	Anna Pollini	45
4.2	Difficoltà incontrate e commenti per i docenti	46
A	Guida utente	47
B	Esercitazioni di laboratorio	48
B.0.1	marco.galli14@studio.unibo.it	48
B.0.2	gianluca.marsella@studio.unibo.it	48
B.0.3	sara.mazzoli8@studio.unibo.it	48

B.0.4	anna.pollini2@studio.unibo.it	48
-------	---	----

Capitolo 1

Analisi

1.1 Descrizione e requisiti

JavaCrush è un gioco single player ispirato al famoso Candy Crush. Lo scopo del giocatore è realizzare dei match, ovvero sequenze di almeno 3 elementi dello stesso tipo, scambiando tra loro celle di una griglia al fine di soddisfare degli obiettivi prestabiliti, entro un determinato numero di mosse. Per mossa si intende uno scambio tra celle adiacenti che genera almeno un match.

Una volta raccolti gli elementi coinvolti nel match, i restanti elementi riempiono i vuoti in base alla gravità applicata, che potrebbe essere diversa in base al livello.

In aiuto del giocatore sono presenti dei power-up, consistenti in pulsanti speciali attivabili una sola volta a partita che permettono di effettuare mosse particolari (esempio: raccoglimento di un'intera riga selezionata).

Durante il corso della partita potrebbero verificarsi casi di stallo, cioè situazioni in cui nessuna mossa porta ad un match. In questi casi gli elementi della griglia vengono rimescolati per creare nuove configurazioni senza stallo.

La partita può terminare con due esiti possibili: vittoria, se si sono soddisfatti gli obiettivi, oppure sconfitta nel caso in cui si terminano le mosse.

Requisiti funzionali:

- Generare in maniera randomica un ambiente di gioco valido.
- Controllare la validità di una mossa.
- In caso di mossa valida raccogliere gli elementi, notificare il progresso agli obiettivi corrispondenti e decrementare le mosse disponibili.
- Aggiornare la griglia con elementi nuovi.
- Controllare situazioni di stallo.
- Monitorare in tempo reale e visualizzare lo stato degli obiettivi e delle mosse per decretare la vittoria/sconfitta.

- Gestione dei power-up: il sistema deve permettere l'attivazione di aiuti speciali una sola volta per sessione, modificando la griglia secondo la logica specifica del potere scelto.

Requisiti non funzionali:

- L'applicazione dovrà garantire un'esperienza di gioco chiara e intuitiva per l'utente.
- Il sistema deve essere modulare ed estendibile (possibilità di aggiungere nuovi tipi di celle, obiettivi e livelli).

1.2 Modello del Dominio

In JavaCrush il giocatore interagisce con una griglia di gioco, composta da celle contenenti oggetti di vario tipo. Il giocatore può effettuare una mossa, ovvero lo scambio di due celle adiacenti della griglia; una mossa è valida solo se produce almeno un match, ossia una sequenza di tre o più oggetti dello stesso tipo allineati. Quando si verifica un match, gli oggetti coinvolti vengono rimossi e la griglia viene aggiornata con nuovi oggetti.

A ogni livello sono associati uno o più obiettivi da soddisfare e un numero massimo di mosse disponibili. Gli obiettivi consistono nella raccolta di un certo numero di oggetti di un tipo specifico: ogni match aggiorna il conteggio degli oggetti raccolti.

Il giocatore dispone inoltre di power-up, ovvero azioni speciali attivabili una volta sola per partita, che consentono di eliminare specifiche celle o intere aree di gioco indipendentemente dalla logica dei match ordinari.

Durante il gioco può verificarsi uno stallo, ovvero una configurazione della griglia in cui nessuna mossa valida è possibile. In tal caso la griglia viene rimescolata.

La partita termina con una vittoria se tutti gli obiettivi sono soddisfatti entro il limite di mosse, o con una sconfitta in caso contrario.

Gli elementi costitutivi del problema sono sintetizzati in figura 1.1

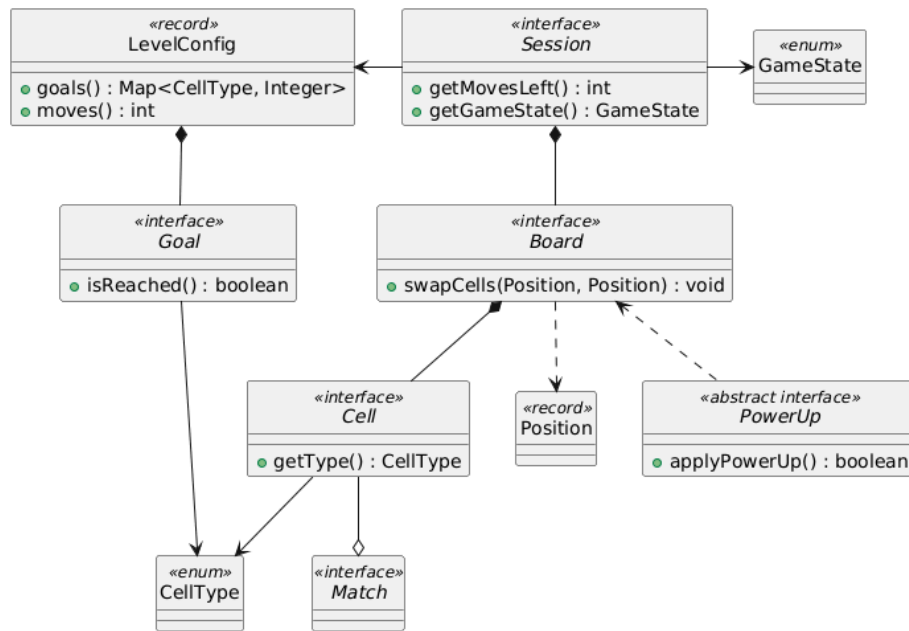


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali e i rapporti fra loro.

Capitolo 2

Design

2.1 Architettura

L'architettura del gioco JavaCrush segue il pattern architetturale MVC, nella sua declinazione "ECB", ossia "entity-control-boundary".

L'interfaccia `GameView` agisce come Boundary del sistema, occupandosi esclusivamente della visualizzazione dello stato di gioco e della cattura degli input dell'utente. Essa non contiene logica di calcolo e interagisce con il `GameController`. Questo, in qualità di Control, funge da orchestratore centrale: esso riceve gli input dalla View, invoca le operazioni necessarie sui componenti del Model e coordina l'aggiornamento della View in risposta ai cambiamenti di stato. Infine, il Model è composto dalle Entity che rappresentano il nucleo logico dell'applicazione, come la griglia, la sessione di gioco e i motori di calcolo.

L'interazione tra queste componenti è studiata per garantire l'indipendenza della logica di dominio rispetto alle scelte grafiche: il Model ignora completamente l'esistenza della `GameView` e non dipende dal framework `JavaFX`.

Mentre la logica di gioco locale alla partita è gestita tramite chiamate dirette tra `GameView` e `GameController`, la gestione della navigazione globale dell'applicazione (come il cambio tra menù, istruzioni e livelli) è delegata all'`AppController`, il quale utilizza un sistema di eventi disaccoppiato per gestire i cambi di stato globali. Questo design garantisce che la sostituzione della View sia un'operazione che non comporta modifiche né ai Controller né al Model, rispettando pienamente i principi di disaccoppiamento richiesti.

In figura 2.1 è esemplificato il diagramma UML architetturale.

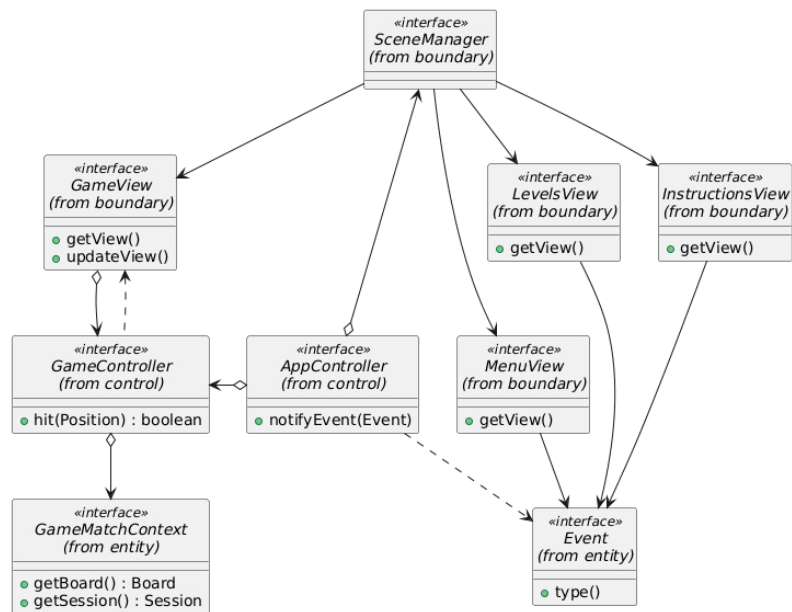


Figura 2.1: Schema UML architetturale di JavaCrush. Le interfacce GameController e AppController fungono da controller del sistema, che coordinano le interazioni tra le View e il Model. GameMatchContext è la classe che fornisce tutte le componenti necessarie al Controller per gestire un determinato livello.

2.2 Design Dettagliato

2.2.1 Marco Galli

Gestione dello stallo della griglia di gioco

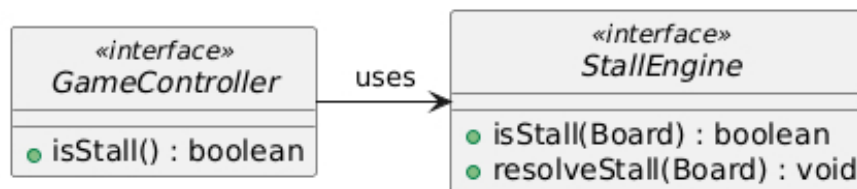


Figura 2.2: Schema UML rappresentante `StallEngine` e la sua interazione con `GameController`.

Problema Durante lo svolgimento della partita possono verificarsi situazioni in cui nessuna mossa è valida (ovvero l'utente, scambiando di posto due celle, non ottenga nemmeno un match): tutte le celle della griglia in questo caso dovranno essere riposizionate in modo casuale fino a far comparire almeno una mossa valida e nessun match già formato.

Soluzione Il compito di rilevare questo stato e, in caso positivo, di risolverlo, è interamente affidato a `StallEngine`. Come mostrato in figura 2.2, per semplificare l'utilizzo di questa classe da parte del Controller è stato creato un metodo pubblico **`resolveStall()`** che controlla se la griglia è in stallo o meno e in caso positivo rimescola le celle. La classe presenta un altro metodo pubblico (**`isStall()`**) che serve al Controller per sapere se si è entrati in stallo: questo metodo viene utilizzato dal Controller principalmente per notificare la View in caso di stallo senza dover ricorrere ad un utilizzo diretto di `StallEngine` da parte della View.

Affidare i compiti di gestione dello stallo ad una sola classe consente di:

- fare chiarezza sui diversi ruoli dei vari componenti del Model
- tracciare una netta linea di separazione tra logica e Controller/View
- rendere il codice riutilizzabile qualora si decidesse di modificare Controller e/o View

Controllo possibili match



Figura 2.3: Schema UML rappresentante la classe `StallEngineImpl` e la sua interazione con `MatchManager`.

Problema Durante il controllo dello stallo, tramite metodi privati, vengono scambiate in maniera fittizia le celle della griglia: si deve poi controllare se questi finti scambi abbiano prodotto dei match.

Soluzione Come si può dedurre dalla figura 2.3, per rilevare la presenza o meno di match possibili si è scelto di avvalersi di un oggetto di tipo `MatchManager`, all'interno della mia classe, che viene utilizzato per rilevare match effettivi sia in fase di controllo che in fase di risoluzione dello stallo: questo ha consentito di riutilizzare codice già scritto e collaudato senza violare la norma di buona programmazione DRY (Don't Repeat Yourself).

Stallo irreversibile

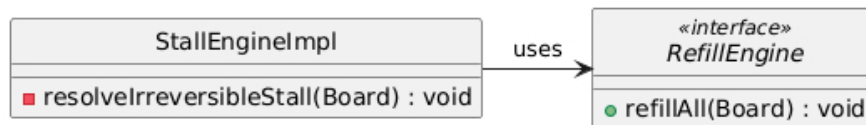


Figura 2.4: Schema UML rappresentante la classe `StallEngineImpl` e la sua interazione con `RefillEngine`.

Problema E' possibile che, con determinate configurazioni di griglia, possa verificarsi un cosiddetto “stallo irreversibile”, ovvero uno stallo particolare che non può essere risolto per mancanza fisica di elementi che portino ad almeno 1 match (per ottenere uno stallo irreversibile è sufficiente che ciascun tipo di cella sia presente nella griglia al più 2 volte, dato che un match viene formato dall'accostamento di almeno 3 celle uguali). Il sistema deve permettere la risoluzione di questo particolare stallo rigenerando tutte le celle della griglia in modo casuale.

Soluzione All'interno dell'applicazione sono state adottate delle strategie tali da impedire alla radice questo particolare stallo impostando le dimensioni della griglia più piccola presente nel gioco (livello “Expert”) ad una dimensione 6x3 (18 celle): avendo a disposizione 6 tipi di celle, il numero di celle della griglia deve essere superiore a 12 per impedire lo stallo irreversibile (6 tipi * al più 2 presenze nella griglia = 12). Nonostante questo, in previsione di future modifiche alle dimensioni delle griglie è stato introdotto un campo statico privato `MAX_ITERATIONS` in `StallEngineImpl` che contiene il numero massimo di rimescolamenti della griglia, superato il quale il siste-

ma rileva uno stallo irreversibile e attiva la funzione di rigenerazione casuale dell'intera griglia avvalendosi di un oggetto `AdaptiveRefill` (per non violare il principio DRY), come mostrato in figura 2.4, il quale contiene il metodo pubblico **`refillAll()`**, il tutto per impedire loop infiniti.

Alternativamente si sarebbe potuto rilevare la presenza dello stallo irreversibile controllando l'intera griglia prima di cercare di risolvere lo stallo, calcolando se, per almeno un tipo, fossero presenti almeno 3 elementi, ma questo avrebbe appesantito il codice con controlli inutili, date le attuali dimensioni minime delle griglie presenti nel gioco.

Suggerimento mossa (Hint)

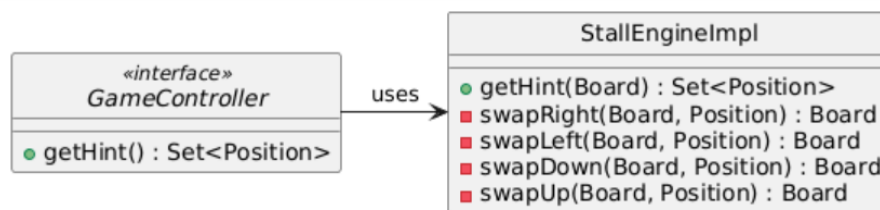


Figura 2.5: Schema UML rappresentante la classe `StallEngineImpl` e la sua interazione con l'interfaccia `GameController`.

Problema Durante il corso della partita l'utente potrebbe trovarsi in difficoltà e non riuscire a scovare nessuna mossa valida da compiere: l'applicazione deve venire incontro a quest'ultimo evidenziando visivamente un qualunque match possibile per poter avanzare nel gioco.

Soluzione Per sopperire a questa necessità di evidenziare in maniera esplicita un possibile match nella griglia è stato progettato un metodo pubblico di `StallEngine` chiamato **`getHint()`** che ritorna un possibile match scelto in modo completamente casuale. Questo metodo è stato inserito all'interno di `StallEngine` per ottimizzare al massimo il codice riutilizzando le funzioni di utilità interne alla classe atte a spostare una cella nelle quattro direzioni (up, down, left, right), precedentemente create per scovare situazioni di stallo. Questo metodo viene utilizzato dal Controller, come mostrato in figura 2.5, per notificare la View riguardo a quali celle della griglia evidenziare o meno.

In alternativa si sarebbe potuta creare una classe a parte con lo scopo di suggerire un possibile match, ma sarebbe risultato svantaggioso in quanto si sarebbero dovute riscrivere alcune funzioni di utilità violando di conseguenza il principio DRY.

Gestione dei power-up

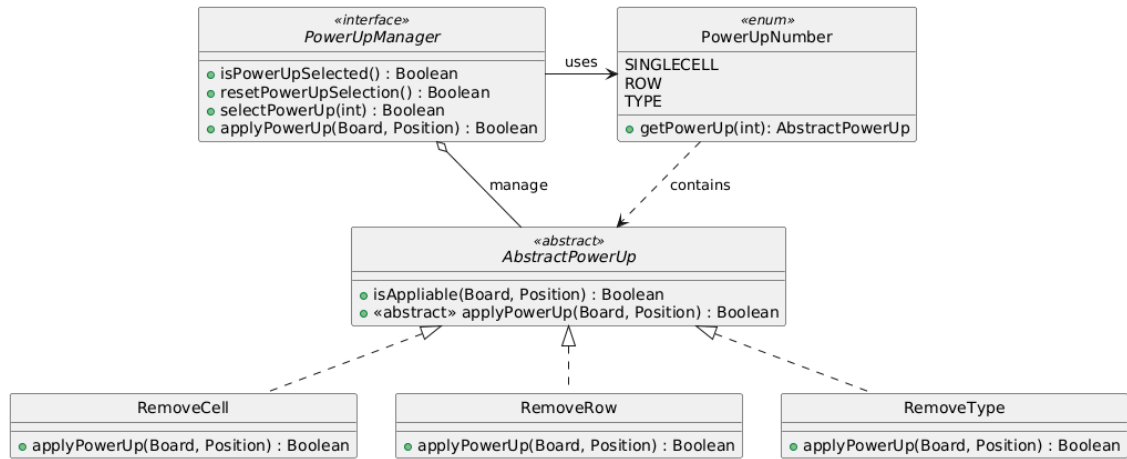


Figura 2.6: Schema UML rappresentante la gestione dei Power-Up tramite classi, interfacce ed enum.

Problema Nel gioco esiste il concetto di Power-Up, ovvero una mossa speciale utilizzabile solo una volta a partita che consente di rimuovere determinate celle dalla griglia. I Power-Up sono di diverso tipo ma già definiti in numero (3). Una volta utilizzato, un Power-Up non può più essere applicato.

Soluzione In questo caso è stata creata una classe astratta (`AbstractPowerUp`) contenente il codice comune a tutti i PowerUp e un metodo astratto (`applyPowerUp()`) che ogni classe PowerUp deve implementare autonomamente, come si nota dalla figura come mostrato in figura 2.6. Per gestire questi PowerUp ho creato una classe `PowerUpManager` che si occupa di gestire la singola applicazione di ogni determinato PowerUp: il concetto di singola applicazione non è di competenza dei singoli PowerUp perché il loro compito è limitato all'effetto che producono, non hanno memoria di quante volte vengono utilizzati. Per mappare ogni singolo PowerUp con un certo "permesso di utilizzo" contenuto nel `PowerUpManager` ho creato una Enum chiamata `PowerUpNumber` dove ogni PowerUp è associato ad un nome simbolico che li identifica. La Enum ha un metodo pubblico `getPowerUp(int)` che prende come parametro il numero (l'indice) dell'`AbstractPowerUp` al quale si vuole accedere e che restituisce l'oggetto PowerUp corrispondente. Questa soluzione di design è stata dettata dal fatto che in caso di aggiunta di nuovi PowerUp la classe `PowerUpManager` non dovrà essere toccata in alcun modo e

le modifiche si limiteranno alla Enum. Alternativamente avrei potuto gestire la scelta del relativo Power Up direttamente all'interno di PowerUpManager, ma sarebbe stata compromessa la scalabilità con future modifiche.

2.2.2 Gianluca Marsella

Gestione dinamica delle regole gravitazionali

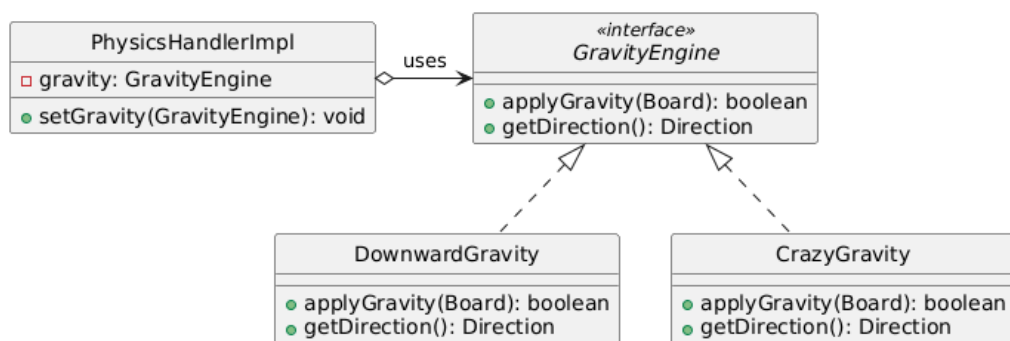


Figura 2.7: Schema UML del pattern Strategy utilizzato per la gestione dinamica delle diverse regole gravitazionali.

Problema Il gioco prevede livelli con diverse regole gravitazionali (caduta verso il basso, verso l'alto, verso sinistra, ecc.) e meccaniche speciali, come la gravità "Crazy", che cambia direzione dinamicamente a runtime quando la scacchiera si stabilizza. Gestire tutte queste logiche condizionali (tramite infiniti if-else) all'interno di un'unica classe del motore fisico avrebbe generato un codice rigido, difficile da mantenere e contrario al principio Open/Closed. Ogni nuova direzione di caduta avrebbe richiesto la modifica del motore centrale.

Soluzione Per risolvere questa criticità si è adottato il *pattern Strategy*, come mostrato in figura 2.7. È stata creata l'interfaccia GravityEngine che espone il metodo **applyGravity()**. Le diverse logiche di caduta implementano questa interfaccia, incapsulando al proprio interno l'algoritmo specifico per quella direzione. In questo design, la classe PhysicsHandlerImpl agisce da Context: essa utilizza un'istanza generica di GravityEngine (tramite composizione) senza conoscerne l'implementazione concreta. Questo disaccoppiamento garantisce un'elevata estensibilità: qualora in futuro si volesse

aggiungere una gravità nuova (ad esempio diagonale), basterà creare una nuova classe che implementi GravityEngine, senza dover alterare in alcun modo il codice del motore fisico.

Riuso del codice nel motore fisico

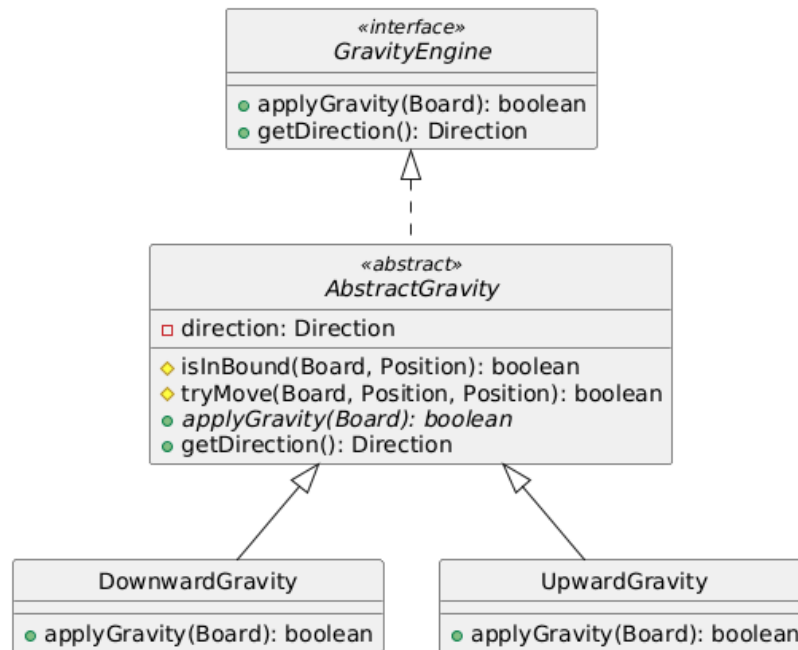


Figura 2.8: Schema UML dell'ereditarietà utilizzata per massimizzare il riutilizzo del codice all'interno del motore fisico (Principio DRY).

Problema Tutte le classi di gravità direzionale condividono la logica di controllo dei confini (**isInBound()**) e la logica di spostamento sicuro di una cella (**tryMove()**). Duplicare queste operazioni in ogni classe (Downward, Upward, ecc.) violerebbe il principio DRY.

Soluzione Per massimizzare il riutilizzo del codice e centralizzare la logica fisica di base, si è introdotta la classe astratta AbstractGravity. Come mostrato in figura 2.8, questa classe implementa l'interfaccia GravityEngine e fornisce alle classi figlie i metodi protetti e condivisi **isInBound()** e **tryMove()**. Grazie a questa architettura, le implementazioni concrete risultano estremamente snelle: esse ereditano i meccanismi di controllo e spostamento, limitandosi a implementare esclusivamente la logica specifica di scorrimento della griglia all'interno del metodo **applyGravity()**.

Semplificazione dell'interazione tra logica di gioco e fisica

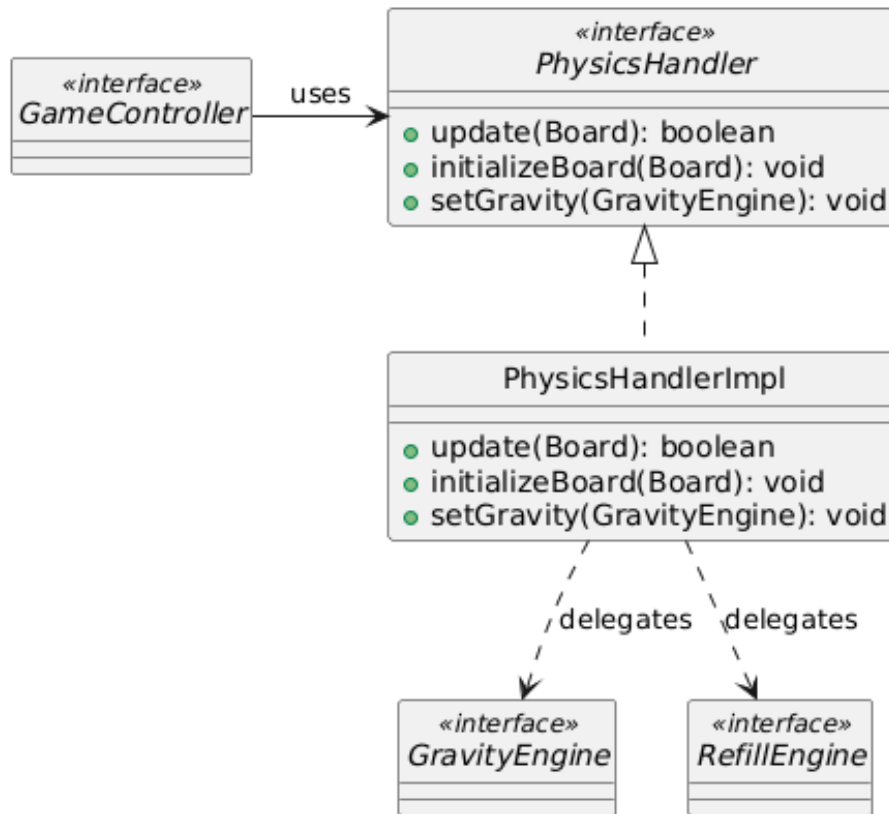


Figura 2.9: Schema UML del pattern Facade utilizzato per incapsulare e orchestrare i sottosistemi fisici.

Problema Durante il corso della partita, il GameController deve poter innescare e gestire l'evoluzione fisica della griglia (come l'applicazione della gravità e il rimpiazzo delle celle esplose). Tuttavia, esporre direttamente al Controller tutti questi motori specifici avrebbe generato un forte accoppiamento. Il Controller si sarebbe ritrovato a dover conoscere e orchestrare l'esatto ordine di esecuzione dei calcoli fisici, violando il principio di singola responsabilità e rendendo il sistema rigido.

Soluzione Si è applicato il *pattern Facade*, come mostrato in figura 2.9. L'interfaccia **PhysicsHandler** funge da facciata unificata, offrendo metodi di alto livello: **update()**, **initializeBoard()** e **setGravity()**. In questo modo, la complessità di coordinare i motori **GravityEngine** e **RefillEngine** è interamente

te incapsulata nel PhysicsHandlerImpl. Il Controller interagisce solo con la Facciata, risultando protetto da eventuali modifiche interne alla logica fisica.

Disaccoppiamento della logica spaziale di riempimento (Astrazione Geometrica)

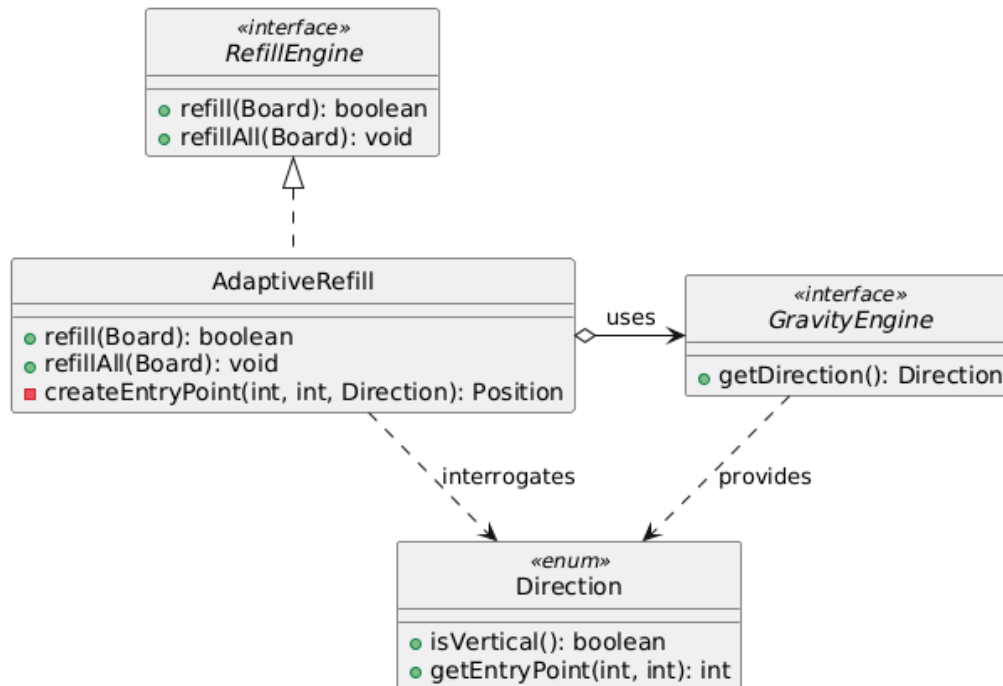


Figura 2.10: Schema UML della delega e dell'astrazione geometrica utilizzate per il riempimento adattivo della griglia.

Problema Il motore di riempimento della griglia (`RefillEngine`) deve sapere esattamente dove far apparire le nuove celle in base alla direzione della gravità (ad esempio, se la gravità tira verso il basso, le celle devono essere generate in alto; se tira verso sinistra, a destra). Gestire queste casistiche spaziali all'interno del motore di refill tramite blocchi condizionali (if-else o switch basati sulle coordinate) avrebbe reso il codice inutilmente prolisso, fragile e fortemente vincolato alle dimensioni fisiche della griglia.

Soluzione Per risolvere il problema, si è scelto di astrarre totalmente la componente spaziale. L'enumerazione `Direction` è stata estesa per includere la logica geometrica di base, esponendo metodi di utilità come `isVertical()` e `getEntryPoint()`, come mostrato in figura 2.10. In questo modo, la classe

AdaptiveRefill è priva di coordinate cablate (hardcoded): per stabilire il punto di generazione delle celle, essa interroga dinamicamente l'oggetto `Direction` associato alla gravità corrente. Questo design garantisce un netto disaccoppiamento: il motore di refill non ha bisogno di conoscere l'orientamento della griglia, ma delega il calcolo delle coordinate all'enumerazione. Così facendo, il sistema è in grado di adattarsi automaticamente a qualsiasi orientamento gravitazionale senza richiedere modifiche alla logica di generazione.

Gestione della creazione dei livelli

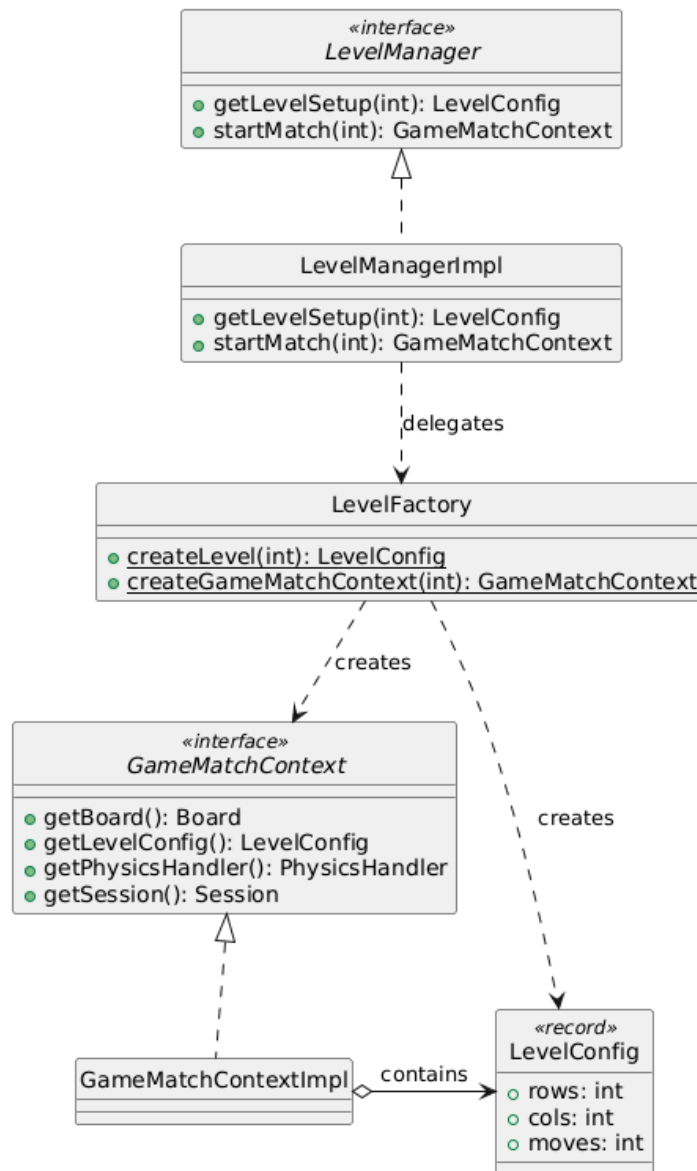


Figura 2.11: Schema UML dei pattern Static/Simple Factory e dell'aggregazione utilizzati per l'inizializzazione sicura del livello.

Problema Durante l'avvio di una partita, il sistema deve istanziare e collegare numerosi moduli di dominio indipendenti (la griglia, il motore fisico, il validatore di mosse, ecc.). Se il GameController ricevesse tutti questi moduli

singolarmente come parametri, subirebbe un difetto architetturale noto come Constructor Over-Injection. D'altro canto, se il LevelManager si occupasse di istanziarli direttamente tramite operatore new, si creerebbe un forte accoppiamento verso le implementazioni concrete. Infine, i parametri statici del livello (numero di mosse, dimensioni, obiettivi) necessitano di una struttura dati semplice e rigorosamente protetta da modifiche accidentali durante la partita.

Soluzione Per risolvere queste tre criticità, si è adottato un approccio architetturale mirato. Per evitare l'over-injection nel Controller, tutti i motori operativi sono stati raggruppati all'interno di un'unica interfaccia aggregatrice (GameMatchContext), che agisce da singolo pacchetto di dipendenze, come mostrato in figura 2.11. Per disaccoppiare il LevelManager dalle implementazioni concrete (come GameMatchContextImpl), l'effettiva istanziazione è stata demandata al *pattern Static/Simple Factory* tramite la classe LevelFactory. Il manager chiama semplicemente i metodi statici della factory, eliminando l'uso diretto dell'operatore new e fungendo solo da intermediario. Infine, per gestire i parametri statici del livello, la factory istanzia e restituisce un Record Java (LevelConfig). Questa struttura dati garantisce compattezza e immutabilità di default, proteggendo la configurazione da qualsiasi alterazione a runtime senza la necessità di scrivere codice ridondante.

2.2.3 Sara Mazzoli

Gestione della validazione delle mosse

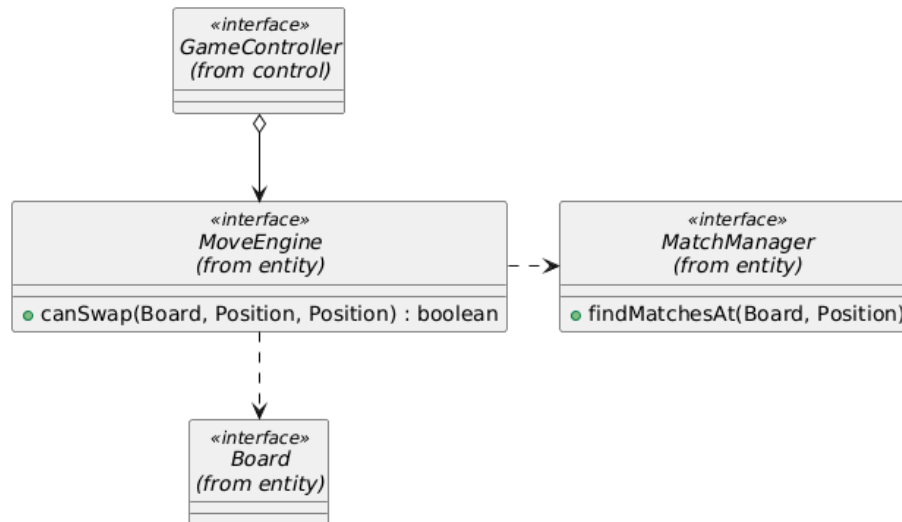


Figura 2.12: Schema UML di MoveEngine e MatchManager per la validazione delle mosse.

Problema Quando il giocatore clicca sulle due celle della griglia di gioco che vuole scambiare, il sistema deve verificare che la mossa sia valida, ovvero che le due celle siano adiacenti (sopra, sotto, sinistra o destra) e che il loro scambio porti effettivamente ad almeno un match

Soluzione Come mostrato in figura 2.12, il compito della validazione è stato delegato all'entità MoveEngine, che fa uso della classe ausiliaria MatchManager. Il Controller invoca il metodo **canSwap()** passandogli come parametro lo stato attuale (Board) e le coordinate. MoveEngine controlla l'adiacenza tra le due celle, e in caso di esito positivo delega a **findMatchesAt()** del MatchManager il compito di verificare se la nuova configurazione produca effettivamente un match, restituendo l'esito.

Gestione dell'esito della validazione della mossa

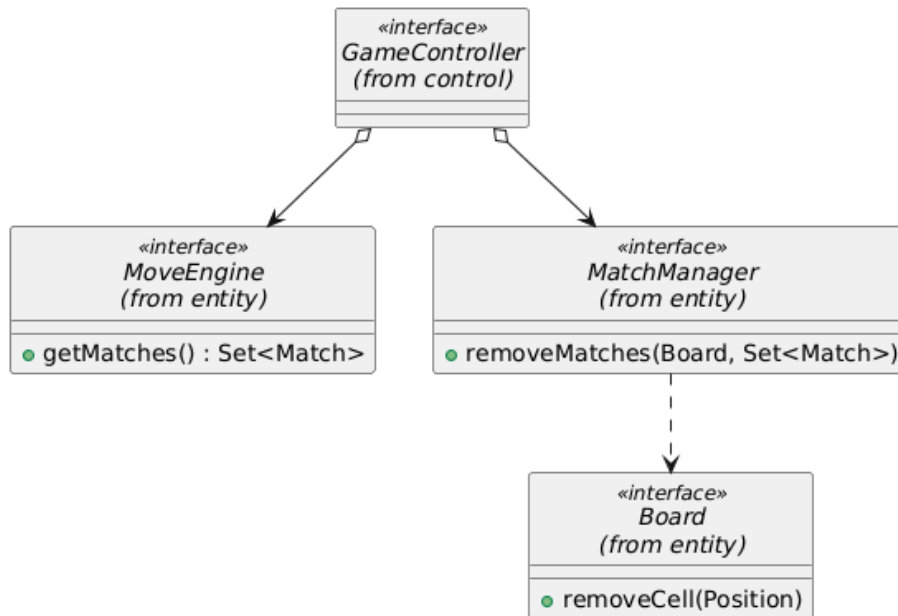


Figura 2.13: Schema UML della gestione dell'esito della validazione e della rimozione fisica delle celle dalla Board.

Problema Una volta stabilito che una mossa è valida, il sistema innesca una serie di effetti a catena. Il problema specifico da risolvere è la rimozione fisica delle celle coinvolte nel match dalla griglia di gioco, operazione che si affianca al parallelo aggiornamento dello stato della partita (goals e mosse) gestito dal resto dell'architettura.

Soluzione Come mostrato in figura 2.13, per fare questo, il GameController si appoggia a MoveEngine e a MatchManager. Per prima cosa, chiede al MoveEngine l'elenco dei match creati, tramite il metodo `getMatches()`, passando quindi le informazioni alle dovute classi per aggiornare goals e diminuire le mosse. Una volta fatto questo usa `removeMatches()` del MatchManager per ripulire i match, il quale provvede a eliminare fisicamente le celle chiamando `removeCell()` della classe Board.

Gestione del Game Loop e sincronizzazione visiva (Boundary e Timeline)

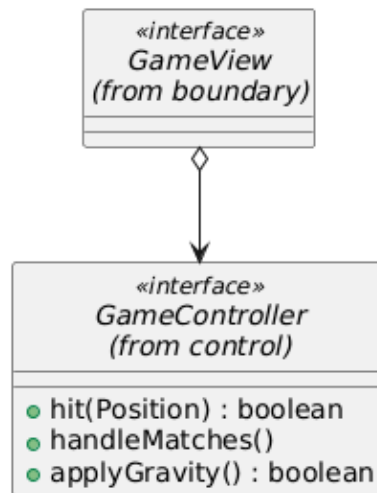


Figura 2.14: Schema UML del Game Loop e della sincronizzazione temporale gestita tramite la *GameView*.

Problema Quando il giocatore esegue una mossa valida, il sistema genera reazioni a catena (eliminazione dei match e successiva caduta delle celle per gravità). Il problema era come scandire temporalmente questi eventi per permettere all'utente di visualizzare fluidamente le animazioni, senza però "sporcare" la classe visiva inserendovi logica di dominio.

Soluzione Come mostrato in figura 2.14, la *GameView* è stata progettata per agire come "orchestratore temporale", pur rispettando il pattern architetturale MVC/ECB nel suo ruolo di Boundary. Al click, la *GameView* verifica la validità dell'azione interrogando il Controller tramite **hit()**. In caso affermativo, essa non calcola l'esito, ma comanda al *GameController* di agire per fasi: prima invoca **handleMatches()** per eliminare le combinazioni, dopodiché sfrutta una Timeline di JavaFX per richiamare ciclicamente il metodo **applyGravity()**, interrompendo il ciclo solo quando il controller segnala la fine della caduta. In questo modo si anticipano cambiamenti futuri: se un domani si volesse accelerare o rallentare il gioco, basterà modificare i tempi della Timeline nella *GameView*, lasciando la fisica del Controller totalmente inalterata.

Disaccoppiamento dell'aggiornamento visivo (Pull Model)

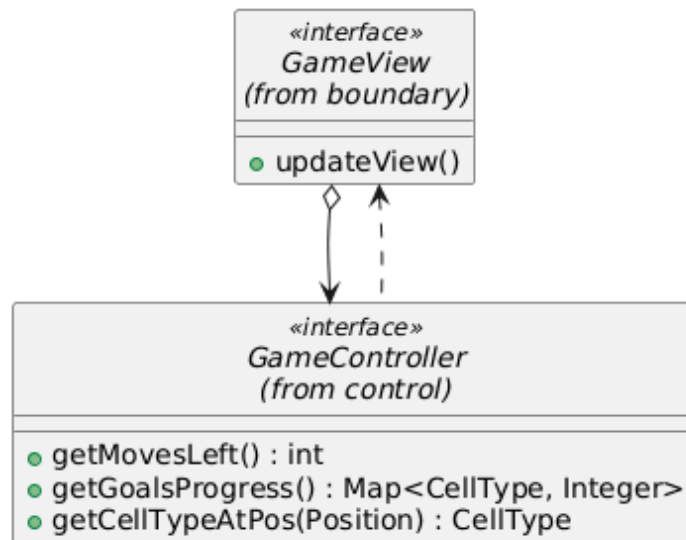


Figura 2.15: Schema UML dell'approccio Pull Model utilizzato dalla GameView per l'aggiornamento visivo.

Problema La schermata di gioco deve aggiornarsi continuamente (mostrando le celle cadute, i nuovi punteggi e le mosse rimaste), ma non deve accedere direttamente alle entità di dominio per evitare un accoppiamento forte e violare l'architettura ECB.

Soluzione Come mostrato in figura 2.15, l'aggiornamento grafico è stato implementato tramite un approccio "Pull". La GameView non ricalcola lo stato del gioco: attende che il controller chiami il metodo **updateView()** per notificare un cambiamento. A quel punto, la GameView interroga attivamente il GameController richiedendo solo i dati aggiornati e necessari per ridisegnare la scena in quel momento.

Inizializzazione del sistema e navigazione centralizzata (Bootstrapping e Routing)

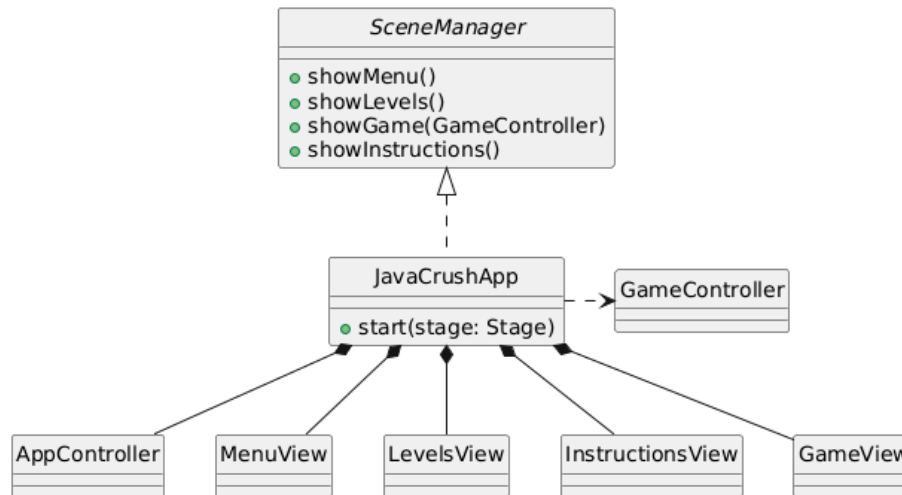


Figura 2.16: Schema UML della navigazione centralizzata e dell'inizializzazione del sistema tramite SceneManager.

Problema All'avvio dell'applicazione, è necessario istanziare e collegare correttamente i componenti dei tre livelli architetturali (MVC/ECB) senza creare un forte accoppiamento. Inoltre, occorre gestire il cambio di schermata (dal Menu al Gioco) in JavaFX in modo fluido, evitando l'apertura di finestre multiple o dipendenze circolari tra le viste.

Soluzione La classe di avvio JavaCrushApp è stata progettata non solo come entry-point, ma come implementazione dell'interfaccia SceneManager, come si può notare in figura 2.16. Durante la fase di avvio (**start()**), la classe si occupa della configurazione delle dipendenze (Dependency Injection), istanziando prima i controller per poi fornirli alle rispettive viste. Per la navigazione, il sistema sfrutta un'unica Scene di cui scambia dinamicamente il nodo radice tramite il metodo **setRoot()** (es. invocato in **showMenu()** o **showGame()**), garantendo transizioni leggere e centralizzate.

Gestione delle comunicazioni globali (Disaccoppiamento tramite Eventi)

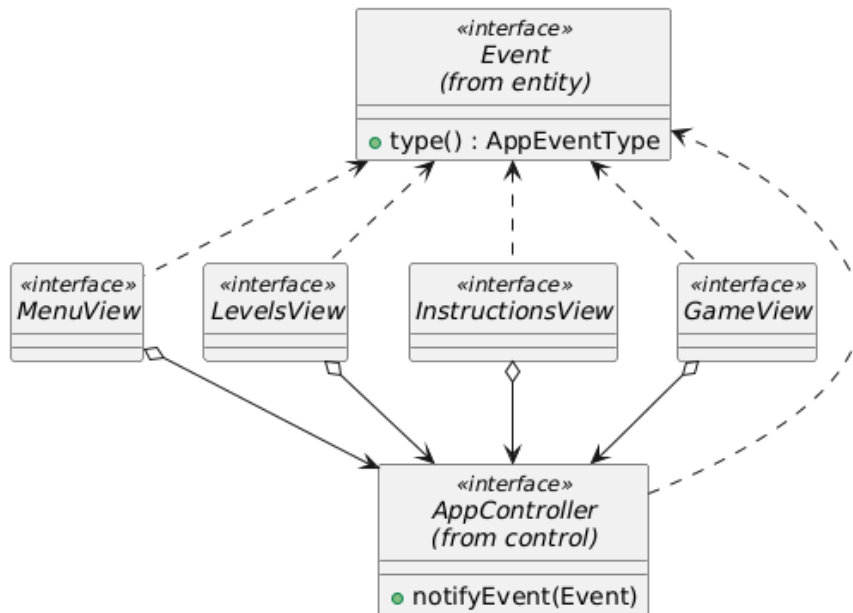


Figura 2.17: Schema UML della comunicazione tramite eventi tra le schermate e il controller principale.

Problema Durante l'utilizzo dell'applicazione, l'utente interagisce con elementi dell'interfaccia che innescano cambiamenti di stato a livello globale, come l'abbandono di una partita in corso, il ritorno al menu principale o l'apertura delle istruzioni. Il problema era permettere alle varie schermate (le View) di comunicare queste intenzioni al gestore centrale senza creare un accoppiamento forte, mantenendole isolate dalle logiche di navigazione esterne al loro specifico dominio.

Soluzione La comunicazione in uscita dalle interfacce è stata standardizzata attraverso un sistema a messaggi. Qualsiasi View agisce come puro Boundary: intercetta l'input dell'utente, istanzia un Event specificandone la natura tramite l'enumerazione AppEventType (es. EXIT_GAME, START_LEVEL) e lo inoltra all'AppController globale utilizzando il metodo **notifyEvent()**, come mostrato in figura 2.17. Un esempio pratico è il metodo **quitLevel()** della GameView, che si limita a confezionare l'evento di chiusura e a delegarlo all'esterno. Questo design garantisce che le classi grafiche rimangano dei semplici messaggeri. Qualora si volessero aggiungere nuove

interazioni globali in futuro, basterà arricchire l'enumerazione degli eventi senza dover modificare le interfacce delle View.

Il design dettagliato dell'infrastruttura globale che riceve, mappa ed elabora questi eventi (*Pattern Command*) è descritto nella sezione a cura di Anna Pollini (paragrafo "Gestione eventi dell'AppController").

2.2.4 Anna Pollini

Creazione dei Goal

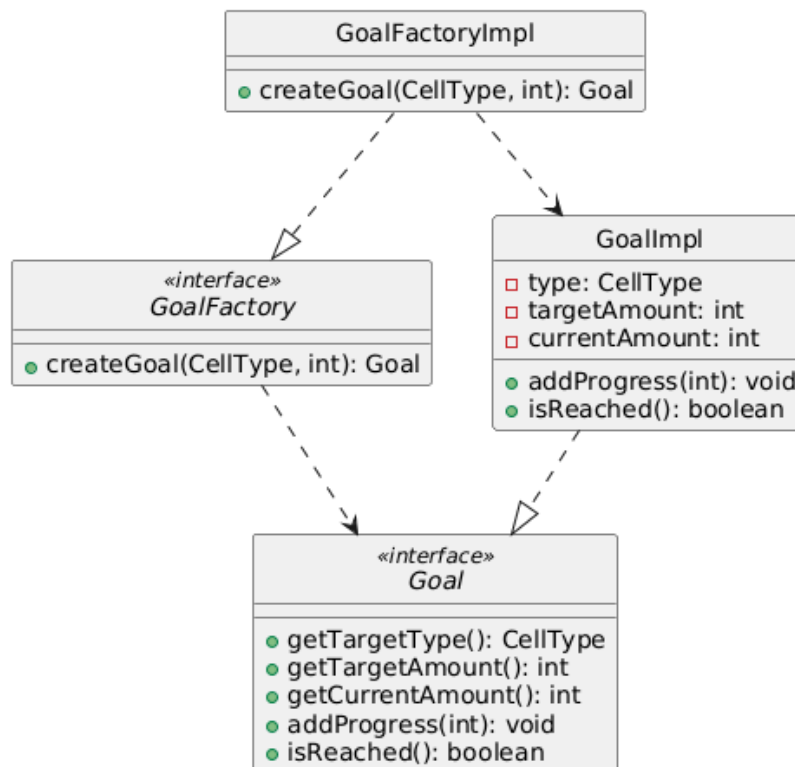


Figura 2.18: Schema UML del pattern Simple Factory utilizzato per la creazione dei goal.

Problema La Session deve gestire una collezione di obiettivi (*Goal*) che variano per tipologia e requisiti. L'istanziamento diretta delle classi concrete all'interno di Session creerebbe un forte accoppiamento tra la logica di gestione del gioco e le specifiche implementazioni dei goal. Ciò porterebbe a

notevoli modifiche del codice di Session qualora nel futuro si volesse inserire una nuova tipologia di obiettivo (ad esempio un obiettivo a tempo).

Soluzione Per risolvere questa criticità è stato applicato il *pattern Simple Factory*, come da figura 2.18. La responsabilità della creazione dei Goal è stata delegata all'interfaccia GoalFactory, la quale con il metodo **createGoal()** incapsula la logica di istanziamento, restituendo alla Session un'astrazione di Goal, anziché una classe concreta. L'uso di un'interfaccia, anziché un metodo statico, evita l'accoppiamento diretto e predispone il sistema a future implementazioni alternative (ad esempio una factory per obiettivi a tempo o per il caricamento da file).

L'implementazione concreta GoalImpl è definita come nested class privata della factory, per nascondere i dettagli implementativi e forzare l'uso dell'interfaccia Goal.

Gestione dello stato della partita

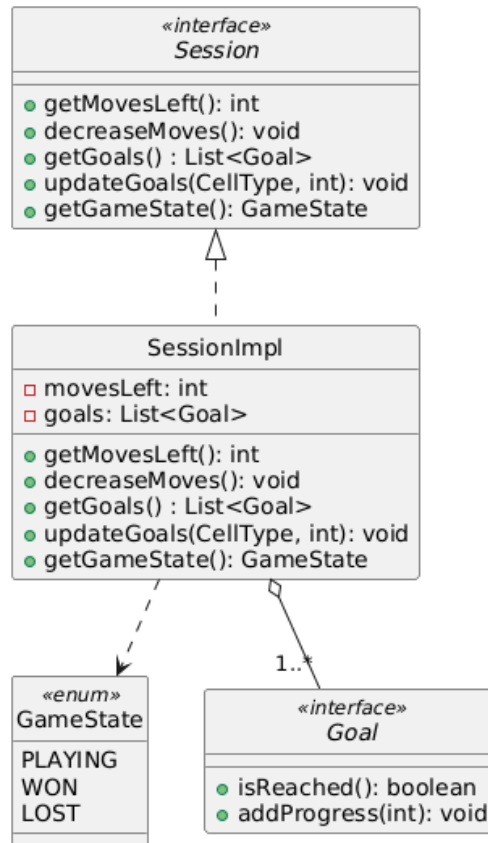


Figura 2.19: Schema UML rappresentante la Session di gioco, con relative interazioni con i goal.

Problema La Session deve essere in grado di decretare autonomamente lo stato della partita (vinta, persa o in corso), senza interagire con Controller o View, seguendo la logica del pattern architetturale MVC. Inoltre deve verificare se gli obiettivi siano stati raggiunti, pur considerando che i criteri di raggiungimento potrebbero variare drasticamente qualora si decidesse di implementare altri tipi di Goal.

Soluzione Come mostrato in figura 2.19, lo stato della partita è gestito tramite l'enum `GameState` con 3 possibili valori: `PLAYING`, `WON` e `LOST`. Lo stato non viene memorizzato come campo, ma ricalcolato ad ogni chiamata di `getGameState()`, in base al numero di mosse residue e allo stato corrente dei goal. I 3 possibili stati sono:

- WON se tutti gli obiettivi sono stati raggiunti (**isReached()**). Questa verifica ha la precedenza per permettere la vittoria anche all'ultima mossa possibile
- LOST se le mosse sono terminate e almeno un goal non è stato completato
- PLAYING altrimenti

Session non implementa controlli condizionali sul tipo di Goal, ma delega la responsabilità a ciascuna istanza tramite il metodo **isReached()**, sfruttando il polimorfismo garantito dall'interfaccia Goal. In questo modo, l'introduzione di nuove tipologie di obiettivo (ad esempio un obiettivo a tempo) non richiede modifiche a Session, poiché ogni nuova implementazione di Goal definirà autonomamente la propria logica di completamento.

Gestione eventi dell'AppController

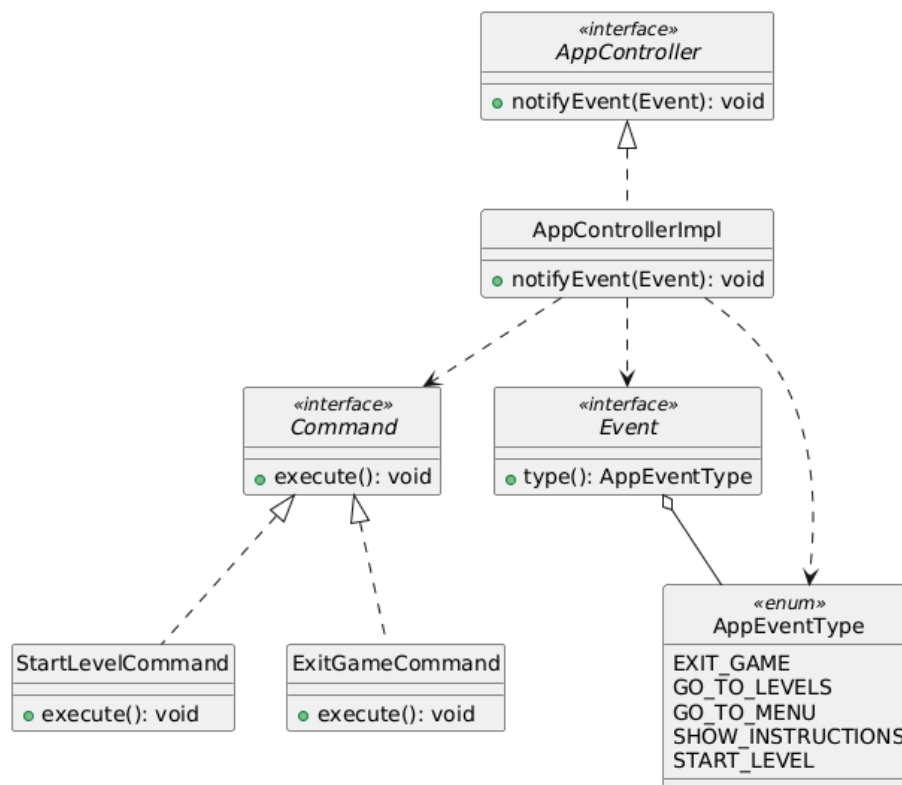


Figura 2.20: Schema UML rappresentante il pattern Command applicato ad AppController.

Problema Un'interazione diretta tra View e ApplicationController creerebbe una forte dipendenza, rendendo difficile modificare l'interfaccia grafica senza impattare la logica di controllo. Inoltre il sistema deve permettere di aggiungere nuove azioni in futuro, senza modificare il Controller esistente, rispettando il principio Open/Closed.

Soluzione È stato introdotto un sistema di comunicazione mediato da oggetti Event, come illustrato in figura 2.20. In questo modo, la View non chiama metodi specifici, ma genera un evento che ne descrive il tipo tramite AppEventType e lo notifica al Controller con **notifyEvent()**. Per gestire ciò è stato utilizzato il *pattern Command*: il Controller legge il tipo dell'evento e, tramite una mappatura interna evento-comando, costruisce ed esegue il Command concreto associato (ad esempio StartLevelCommand o ExitGameCommand), senza conoscerne i dettagli implementativi. L'aggiunta di nuove funzionalità (ad esempio un comando "Undo") non impatta la logica esistente del Controller, ma richiede solamente una nuova classe concreta che implementa Command e la sua registrazione nella mappatura.

Gestione della logica di gioco

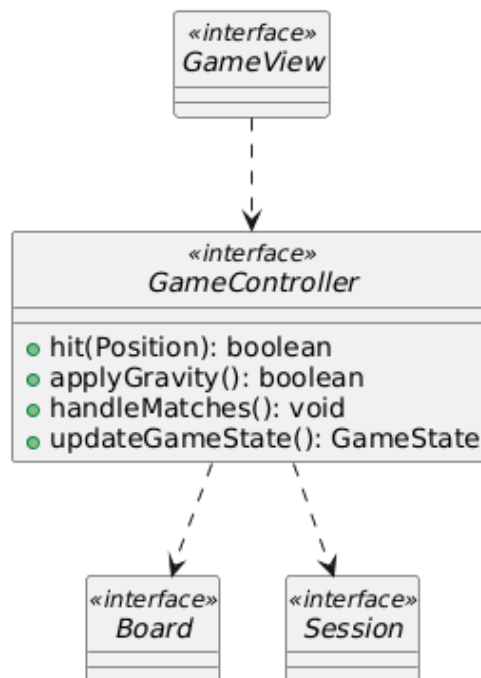


Figura 2.21: Schema UML rappresentante il GameController.

Problema Oltre a coordinare Model e View secondo MVC, il Controller deve gestire la sequenza delle azioni di una singola mossa: una mossa valida innesca una sequenza di eventi, ovvero la caduta degli elementi, la ricerca di nuovi match e la loro rimozione, ed infine l'aggiornamento degli obiettivi. Ciò si ripete finché la griglia non torna stabile, prima di poter accettare un nuovo input dall'utente. È necessario garantire che questa sequenza avvenga nell'ordine corretto, evitando che lo stato del Model venga letto o modificato mentre è ancora incoerente.

Soluzione Per risolvere questo problema, nel GameController sono stati creati metodi specifici per ciascuna fase della mossa valida (**hit()**, **applyGravity()**, **handleMatches()** e **updateGameState()**), come rappresentato in figura 2.21. Sarà la GameView a gestire la sequenza nel tempo, mentre il Controller garantisce che ciascuna fase agisca sempre su uno stato del Model coerente con quella precedente, coordinando al suo interno le componenti coinvolte (ad esempio Board per la griglia e Session per gli obiettivi).

In questo modo, la GameView non accede mai direttamente al Model, ma osserva l'evoluzione del gioco esclusivamente attraverso le risposte del Controller a ogni fase.

Capitolo 3

Sviluppo

3.1 Testing automatizzato

3.1.1 Marco Galli

- **StallEngineTest** Si è testato il mancato apporto di modifiche alla griglia qualora almeno 1 cella fosse vuota e nel caso in cui fossero presenti dei possibili match, si è testato il rimescolamento della griglia con conseguente apparizione di almeno 1 mossa possibile e, dunque, la scomparsa della situazione di stallo. Infine è stato testato il meccanismo di aiuto (Hint) controllando che il risultato fornito dal metodo **getHint()** fosse uno tra quelli possibili elencati (5 possibili mosse differenti).
- **PowerUpManagerTest** Si è testato il controllo di selezione del PowerUp (se c'è attualmente un PowerUp selezionato o meno), l'im-

possibilità di selezionare un PowerUp non compreso tra i 3 possibili elencati, la possibilità di selezionare un PowerUp esistente e la deselectione di un PowerUp. Sono stati testati i 3 PowerUp, per ognuno in particolare: l'impossibilità di applicare un PowerUp quando non ne è stato selezionato ancora uno, la mancanza di Match formati prima di applicare un PowerUp, la possibilità di selezionare lo stesso PowerUp più di una volta ma l'impossibilità di applicarlo nuovamente dopo averlo già usato, la deselectione del PowerUp e il successivo tentativo di utilizzo impedito da PowerUpManager. Oltre a questi test in comune a tutti i PowerUp è stata controllata la correttezza dei Match formati (ovvero delle celle rimosse dalla griglia) dopo aver applicato il PowerUp, in modo da risultare corretti: per SINGLECELL (Hammer, ovvero RemoveCell) un Match contenente una sola cella, per ROW (Rocket, ovvero RemoveRow) un Set<Match> contenente un Match differente per ogni tipo di cella rimossa (riunendo dentro ad uno stesso Match tutte le celle dello stesso tipo), per TYPE (Magic bomb, ovvero RemoveType) un unico Match che racchiude tutte le celle dello stesso tipo raccolte dalla griglia.

3.1.2 Gianluca Marsella

- **AbstractGravityTest** (e implementazioni direzionali) Si è testata la corretta applicazione della logica di spostamento per le diverse direzioni fisse di caduta (verso il basso, l'alto, destra e sinistra). Al fine di massimizzare il riuso del codice e rispettare il principio DRY anche in fase di testing, è stata implementata la classe astratta AbstractGravityTest, che verifica i comportamenti geometrici comuni a tutte le direzioni: lo scorrimento libero di una cella, il blocco in presenza di ostacoli e l'arresto corretto al raggiungimento dei bordi della griglia.
- **CrazyGravityTest** Si è validato il comportamento della gravità dinamica. I test verificano la robustezza della classe accertando che venga lanciata un'IllegalArgumentException qualora il motore venga inizializzato con una lista vuota di strategie. Inoltre, si è verificata la corretta applicazione delle regole di gameplay: il cambio di direzione deve avvenire rigorosamente solo quando la griglia raggiunge uno stato di totale stabilità (nessun movimento rilevato), mentre la direzione corrente deve rimanere inalterata fintanto che le celle continuano a scorrere.
- **PhysicsHandlerTest** Si è testato il motore di orchestrazione fisica, garantendo la corretta inizializzazione della griglia. In particolare, è

stato validato il meccanismo di "flusso continuo" (ovvero la caduta delle celle e il loro immediato rimpiazzo all'interno della stessa iterazione). È stato inoltre accertato l'effettivo raggiungimento di uno stato stabile della griglia implementando un contatore di sicurezza per scongiurare loop infiniti, e si è verificato il corretto riadattamento spaziale della griglia a seguito del cambio dinamico della direzione gravitazionale.

- **RefillEngineTest** Si è testato il comportamento spaziale del riempimento adattivo. I test verificano che le nuove celle vengano introdotte nella griglia a partire dai bordi corretti in stretta dipendenza dall'attuale vettore gravitazionale (ad esempio, immissione dall'alto se la gravità spinge verso il basso). È stato inoltre accertato che il motore non alteri inutilmente lo stato della board qualora questa risulti già completamente piena, e che il metodo di rimpiazzo totale riempia efficacemente tutti gli spazi vuoti durante il setup.
- **LevelFactoryTest** Si è validata la robustezza del processo di creazione dei livelli. I test verificano che le configurazioni prodotte (LevelConfig) rispettino rigorosamente i vincoli strutturali del gioco, come dimensioni minime della griglia, numero di mosse strettamente positivo e presenza effettiva di obiettivi. Inoltre, è stato testato l'assemblaggio del GameMatchContext, accertando che tutti i moduli operativi (motore fisico, validatore mosse, ecc.) vengano istanziati senza riferimenti nulli e presentino una perfetta coerenza dimensionale con la griglia.
- **LevelManagerTest** Si è testato il corretto avvio di una partita. Il test verifica che il manager, a seguito della richiesta di inizio di uno specifico livello, funga correttamente da intermediario e restituisca un contesto di gioco (GameMatchContext) non nullo e completamente inizializzato, assicurandosi che i parametri statici del livello (come le mosse a disposizione) vengano caricati coerentemente con quanto definito in fase di progettazione.
- **CellTest** Si è validata l'implementazione della classe Cell, verificando che la creazione di ogni cella avvenga correttamente attraverso l'assegnazione del relativo CellType. I test assicurano la robustezza del costruttore, confermando il corretto sollevamento di una NullPointerException nel caso di parametri non validi. È stata inoltre verificata la coerenza del metodo **toString()**, garantendo che la rappresentazione testuale della cella sia sempre conforme al formato atteso, facilitando così il debug e la tracciabilità dello stato dei componenti all'interno del modello.

3.1.3 Sara Mazzoli

- **MatchManagerTest** Si è testata la solidità dell'algoritmo di ricerca dei match. Si è validata la corretta identificazione di sequenze standard (orizzontali e verticali) e di configurazioni complesse come forme a "L" e a "T", che richiedono una navigazione multidirezionale della griglia. I test coprono inoltre casi limite, come la presenza di match parziali che non soddisfano il criterio di lunghezza minima, garantendo l'assenza di falsi positivi. Infine, è stata verificata l'efficacia del processo di rimozione fisica delle celle, accertando che il MatchManager agisca correttamente sullo stato della Board svuotando le posizioni coinvolte.
- **MoveEngineTest** Si è testato il comportamento del motore di movimento, focalizzandosi sulla correttezza delle regole di gameplay. I test verificano non solo la logica di adiacenza, accertando che il sistema rifiuti scambi tra celle non adiacenti, ma anche la validazione della mossa: **canSwap()** restituisce true solo se lo scambio produce effettivamente una configurazione vincente (match). Il test **testGetMatches()** è particolarmente significativo poiché simula una mossa complessa che innesca molteplici combinazioni simultanee, garantendo che il MoveEngine sia in grado di mappare correttamente tutti i match risultanti e di restituirli in una struttura dati coerente, pronto per essere consumato dal Controller per l'aggiornamento dello stato di gioco.
- **MatchTest** Si è testata la logica di definizione e gestione delle sequenze vincenti all'interno del gioco. I test validano la corretta creazione di un oggetto Match, assicurando che la struttura associ correttamente il set di posizioni occupate dalle celle coinvolte e il relativo CellType. È stata inoltre verificata la gestione di scenari limite, come la creazione di un match vuoto, confermando che il sistema identifichi correttamente il caso in cui non siano presenti celle all'interno della sequenza.
- **BoardTest** Si è verificata l'integrità strutturale della griglia di gioco, assicurando che la Board inizializzi correttamente le proprie dimensioni. I test validano le operazioni fondamentali di gestione del modello, ovvero l'inserimento e il recupero delle celle tramite coordinate specifiche, gestendo sia le posizioni occupate che quelle vuote (tramite Optional). È stata testata in modo esaustivo la logica di **swapCells()**, verificando che lo scambio tra due celle avvenga correttamente anche in scenari di interazione con posizioni vuote. Infine, è stata confermata la corretta implementazione del metodo **removeCell()**, garantendo che

l'operazione di rimozione svuota effettivamente lo stato della cella alla posizione indicata, mantenendo la coerenza dell'intera struttura dati.

3.1.4 Anna Pollini

- **GoalTest** Si è testata la corretta inizializzazione e la coerenza dello stato di un obiettivo (Goal) subito dopo la sua creazione tramite la factory, verificando che il tipo di cella target e il quantitativo richiesto corrispondessero ai parametri forniti e che l'obiettivo non risultasse inizialmente completato. È stato verificato il funzionamento del metodo **addProgress()**, controllando che il questo aggiorni correttamente la quantità corrente sia per un valore minore del target iniziale, sia per un valore maggiore (evitando stati incoerenti), e che il tentativo di inserire un valore negativo scateni un'eccezione di tipo `IllegalArgumentException`. Infine si è testato il metodo **isReached()**, accertando che l'obiettivo risulti completato solo al raggiungimento del target iniziale prestabilito.
- **SessionTest** Si è testata la corretta inizializzazione della Session del gioco, verificando il caricamento del numero di mosse disponibili, l'immutabilità della lista dei goal restituiti (lanciando un'eccezione del tipo `UnsupportedOperationException` qualora questa lista provasse ad essere modificata) e la coerenza dello stato iniziale impostato su `PLAYING`. È stato testato il meccanismo di decremento delle mosse tramite il metodo **decreaseMoves()**, verificando inoltre che il tentativo di decrementare le mosse quando queste sono già esaurite provochi un `IllegalStateException`. Per quanto riguarda l'avanzamento del gioco, si è testato il metodo **updateGoals()** verificando che l'aggiornamento incrementi solo i goal associati al `CellType` raccolto, lasciando invariati gli altri obiettivi. Infine, sono stati testati i criteri di calcolo del `GameState`: si è accertato che la sessione decreti lo stato di `WON` non appena tutti i goals risultano completati (anche in presenza di mosse residue) e lo stato di `LOST` qualora il contatore delle mosse si azzeri prima del totale soddisfacimento degli obiettivi previsti.
- **AppControllerTest** Utilizzando Mockito, si sono testati i comandi associati agli eventi gestiti dall'`AppControllerImpl`, ovvero `ExitGameCommand`, `GoToMenuCommand`, `GoToLevelsCommand`, `ShowInstructionsCommand` e `StartLevelCommand`. Per il test di quest'ultimo metodo, è stato creato un `GameMatchContext` fittizio (configurato con i determinati moduli mock) in modo da poter simulare il comportamento del `LevelManager`. Il test verifica che il riferimento al `GameController`

corrente passi da vuoto a presente a seguito della notifica dell'evento, e che venga invocato correttamente **showGame()** sullo SceneManager con l'istanza del Controller appena creata.

- **GameControllerTest** Utilizzando Mockito, si sono testati i metodi che compongono il ciclo di una mossa. Il metodo **hit()** è stato verificato in entrambe le sue azioni: **testNormalHit()** verifica il corretto scambio di due celle adiacenti e il decremento delle mosse, mentre **testPowerUpHit()** verifica che, una volta selezionato un power-up, il Controller deleghi l'azione al gestore del power-up e ne restituisca l'esito. Inoltre è stato verificato il comportamento del metodo **applyGravity()** in tre scenari distinti: quando la griglia cambia per effetto della gravità, quando la griglia è stabile e non presenta match (con la chiamata alla logica di risoluzione dello stallo), e quando la griglia è stabile ma vengono trovati nuovi match da gestire.

3.2 Note di sviluppo

Di seguito vengono analizzate in dettaglio le principali funzionalità avanzate del linguaggio Java e delle sue librerie, documentando le ragioni d'uso e la loro collocazione nel codice sorgente:

3.2.1 Marco Galli

- **Lambda Expressions**
Utilizzate principalmente in `GameViewImpl`, per esempio per la creazione di un `KeyFrame`, per la gestione di un `Alert` e di un `Service<Integer>`.
PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/view/impl/GameViewImpl.java#L291>
PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/view/impl/GameViewImpl.java#L436>
PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/view/impl/GameViewImpl.java#L481>
- **Stream**
Utilizzati in `RemoveRow` per suddividere le celle raccolte in diversi `Match`, uno per ogni tipo, utilizzati in `StallEngineImpl` per filtrare `Match` doppi o nulli.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/powerup/impl/RemoveRow.java#L32>

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/model/impl/StallEngineImpl.java#L213>

- **JavaFX.concurrent**

Utilizzate le classi `Service` e `Task` per implementare un timer in background che allo scadere mostra una possibile mossa sullo schermo.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/view/impl/GameViewImpl.java#L442-L472>

Per creare il timer utilizzando le classi `Service` e `Task` ho preso spunto dalla documentazione API di JavaFX 26 (<https://openjfx.io/javadoc/26/javafx.graphics/javafx/concurrent/Task.html>), in particolare l'esempio "A Simple Loop With Progress Notification" (<https://openjfx.io/javadoc/26/javafx.graphics/javafx/concurrent/Task.html#a-simple-loop-with-progress-notification-heading>), e da un video su YouTube postato dal canale Bro Code (<https://www.youtube.com/watch?v=SztE5W41on4&t=2s>) riguardante il tema dei Threads, il tutto adeguatamente rivisitato e adattato al contesto della nostra applicazione.

3.2.2 Gianluca Marsella

- **Optional**

Utilizzato in `AbstractGravity` e `AdaptiveRefill` per gestire la presenza di celle nella griglia in modo sicuro, evitando riferimenti null e rendendo esplicita la gestione delle celle vuote tramite metodi come `isPresent()` o `isEmpty()`.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/60d789307094acff929a7c970929bb3161457c3e/src/main/java/it/unibo/javacrush/model/impl/gravity/AbstractGravity.java#L71>

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/60d789307094acff929a7c970929bb3161457c3e/src/main/java/it/unibo/javacrush/model/impl/AdaptiveRefill.java#L58>

- **Stream**

Utilizzata in `AdaptiveRefill` per processare in modo dichiarativo le coordinate della griglia durante il rimpiazzo delle celle, riducendo la complessità rispetto ai cicli for annidati e migliorando la leggibilità del

codice.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/60d789307094acff929a7c970929bb3161457c3e/src/main/java/it/unibo/javacrush/model/impl/AdaptiveRefill.java#L40-L44>

- **Lambda Expressions**

Utilizzate in `AdaptiveRefill` per definire in modo conciso le logiche di filtraggio delle posizioni nella griglia, rendendo il codice più leggibile e funzionale rispetto all'uso di cicli tradizionali.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/60d789307094acff929a7c970929bb3161457c3e/src/main/java/it/unibo/javacrush/model/impl/AdaptiveRefill.java#L41-L43>

- **Dichiarazione di codice e fonti**

Per applicare il pattern Facade si è fatto uso delle slide del prof Piani e dal sito Refactoring.Guru (<https://refactoring.guru/design-patterns/catalog>).

3.2.3 Sara Mazzoli

- **Optional e Functional Style**

Utilizzato in `MatchManagerImpl` per gestire in modo dichiarativo la presenza delle celle, evitando controlli null espliciti e migliorando la robustezza del codice.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/model/impl/MatchManagerImpl.java#L81-L85>

- **Stream**

Utilizzato in `MatchManagerImpl` per filtrare e mappare i tipi di cella in modo dichiarativo.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/model/impl/MatchManagerImpl.java#L81-L85>

- **Lambda Expressions**

Utilizzate in

- `GameViewImpl` per definire le azioni dei bottoni (es. `setOnAction`).

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/>

blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/
it/unibo/javacrush/view/impl/GameViewImpl.java#L327-L345

- MatchManagerImpl per definire logiche di filtro in modo conciso.
PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/model/impl/MatchManagerImpl.java#L81-L85>

- **JavaFX**

Utilizzata per la costruzione delle View, per esempio GameView.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/view/impl/GameViewImpl.java#L78-L94>

- **EnumMap**

Utilizzata per associare in modo efficiente e type-safe i tipi di cella alle rispettive immagini di gioco, ottimizzando l'accesso alle risorse durante il rendering della griglia.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/view/impl/GameViewImpl.java#L109-L121>

- **Timeline e KeyFrame**

Utilizzato per scandire temporalmente le fasi della mossa (eliminazione match e gravità), garantendo che l'interfaccia grafica rimanga coerente durante le transizioni senza bloccare il thread principale.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/view/impl/GameViewImpl.java#L289-L316>

- **Platform.runLater**

Utilizzato in GameViewImpl per sincronizzare in modo sicuro l'aggiornamento dell'interfaccia (es. avvisi di vittoria o sconfitta) quando questi eventi vengono scatenati da logiche di controllo asincrone, evitando conflitti.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/view/impl/GameViewImpl.java#L308>

3.2.4 Anna Pollini

- **Optional**

È stato utilizzato in:

- Event e GameEvent per gestire il campo id del livello, necessario solo per l'evento START_LEVEL e assente negli altri, evitando l'uso di null per rappresentare un valore opzionale.
PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/controller/api/Event.java#L24>
PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/common/GameEvent.java#L15>
- ApplicationController (e relativa implementazione ApplicationControllerImpl) per tracciare il riferimento al GameController corrente, presente solo durante una partita attiva.
PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/controller/api/AppController.java#L24>
PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/controller/impl/AppControllerImpl.java#L60-L62>

- **Functional Interfaces e Lambda Expressions**

Sono state utilizzate in:

- ApplicationControllerImpl, in cui la mappa interna Map<AppEventType, Function<Event, Command>> associa ogni tipo di evento a una Function che genera il comando corrispondente, tramite l'utilizzo combinato di interfacce funzionali e lambda.
PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/controller/impl/AppControllerImpl.java#L27>
PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/controller/impl/AppControllerImpl.java#L72>
- StartLevelCommand, con il Consumer<GameController> nel costruttore che consente di implementare un meccanismo di callback che comunica il GameController appena creato al chiamante senza creare dipendenze dirette verso ApplicationControllerImpl.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/controller/impl/commands/StartLevelCommand.java#L21>

- SessionImpl, con l'uso di lambda nel costruttore per mappare i requisiti di ingresso e nel metodo updateGoals() per aggiornare selettivamente i goal corrispondenti al tipo di cella raccolto.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/model/impl/SessionImpl.java#L34>

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/model/impl/SessionImpl.java#L70-L74>

- **Stream**

Sono stati utilizzati in:

- GameControllerImpl nel metodo handleMatches(), per iterare su un Set di Match in modo da poter aggiornare i goals e successivamente rimuovere le celle coinvolte.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/controller/impl/GameControllerImpl.java#L80-L84>

- SessionImpl nel costruttore, per mappare la mappa in ingresso in obiettivi concreti, e nel metodo isGameWon() per la verifica del completamento di ciascun goal.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/model/impl/SessionImpl.java#L33-L35>

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/main/java/it/unibo/javacrush/model/impl/SessionImpl.java#L96-L97>

- **Librerie esterne**

Mockito: framework utilizzato per i test del controller, ovvero nelle AppControllerTest e GameControllerTest, in modo da poter isolare i componenti logici ed effettuare i relativi test.

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/test/java/it/unibo/javacrush/controller/AppControllerTest.java#L40-L41>

PERMALINK: <https://github.com/Marselo1010/OOP25-JavaCrush/>

blob/bd23551415323be34d50fc64c9f79ace76f3fd9a/src/test/java/it/unibo/javacrush/controller/GameControllerTest.java#L37-L38

- **Dichiarazione di codice e fonti**

Per comprendere e applicare il pattern Command sono state consultate le seguenti risorse:

- GeeksforGeeks: <https://www.geeksforgeeks.org/system-design/command-pattern/>
- Medium: <https://medium.com/@akshatsharma0610/command-design-pattern-in-java-1b8bbb699ff7>
- StackOverflow: <https://stackoverflow.com/questions/49036887/command-pattern-with-additional-argument-parsing>

Le soluzioni descritte nelle prime due risorse sono state analizzate come riferimento per la struttura del pattern (interfaccia Command con metodo execute() e classi concrete per ogni azione). Invece la pagina di StackOverflow ha ispirato l'utilizzo di una mappa (Map<AppEventType, Function<Event, Command>>) per associare ogni tipo di evento al proprio comando, permettendo in questo modo di aggiungere nuovi comandi registrando una nuova entry nella mappa. L'architettura risultante è stata riprogettata e rimodellata per integrarsi nel contesto e nei requisiti specifici del dominio del videogioco.

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Marco Galli

Nel progetto mi sono occupato principalmente dello stallo della griglia, ovvero come intercettarlo ed eventualmente risolverlo a livello logico, ho creato e gestito il meccanismo della Hint (il suggerimento mossa) a 360 gradi, dalla logica fino alla grafica, infine ho creato la logica dei Power-Up, ossia la loro creazione, il loro utilizzo e il loro effetto prodotto.

- **Punti di forza:** il codice da me progettato impedisce un accesso diretto ai singoli PowerUps, che è invece possibile solo attraverso il PowerUpManager con l'ausilio di PowerUpNumber, supporta eventuali futuri ampliamenti con la creazione di nuovi PowerUp in maniera veloce ed efficace con la semplice modifica di PowerUpNumber, suddivide in modo chiaro e preciso i compiti delle varie classi e accorpa dentro a StallEngine tutte le funzionalità relative allo stallo e alle mosse possibili (Hint compresa).
- **Punti di debolezza:** certe parti di codice all'interno di alcuni metodi (primo fra tutti `getHint()` di StallEngine) sarebbero potute essere state scritte in maniera più chiara e meno pesante per aiutare anche la comprensione a livello visivo delle funzionalità da loro eseguite da parte di altri sviluppatori.

4.1.2 Gianluca Marsella

Nel progetto mi sono occupato dell'intera infrastruttura del motore fisico del gioco, progettando e implementando la gestione dinamica della gravità, il motore di riempimento della griglia (RefillEngine) e il sistema di inizializzazione dei livelli.

- **Punti di forza:** L'architettura del motore fisico è caratterizzata da un elevato grado di estensibilità grazie all'adozione del pattern Strategy per le regole gravitazionali e al pattern Facade per l'interazione con il Controller, che hanno permesso di isolare la complessità computazionale dal flusso di gioco. L'implementazione di una gerarchia basata su classi astratte (AbstractGravity) ha favorito il riuso del codice (rispetto del principio DRY), mentre l'uso di Record (LevelConfig) e Optional ha garantito un'architettura dati immutabile e sicura. Infine, il sistema di riempimento adattivo, basato sull'astrazione geometrica tramite l'enumerazione Direction, rende il motore capace di adattarsi a qualsiasi variante gravitazionale senza richiedere modifiche al codice core.
- **Punti di debolezza:** La sfida logica più complessa è stata l'implementazione di CrazyGravity. A differenza delle gravità con direzione fissa, questa richiede una gestione dello stato della scacchiera che deve essere verificata in modo asincrono: garantire che il cambio di direzione avvenga solo dopo la totale stabilità della griglia ha introdotto una logica di controllo più articolata rispetto al resto del motore fisico. Inoltre, la scelta di mantenere ogni parte del motore separata in

componenti molto piccoli ha creato un sistema con un'elevata scomposizione; sebbene questo abbia garantito un ottimo disaccoppiamento, ha richiesto uno sforzo di integrazione iniziale non trascurabile per allineare correttamente il mio lavoro con le componenti sviluppate dal resto del gruppo.

4.1.3 Sara Mazzoli

Nel progetto mi sono occupata della gestione delle mosse e dei match. Nello specifico nel motore di validazione di esse, dell'algoritmo di ricerca dei match e delle loro eliminazione dalla griglia di gioco. Mi sono inoltre occupata dell'implementazione delle diverse interfacce grafiche dell'applicazione e della logica di navigazione (Scene Manager), gestendo il caricamento dinamico di menù, istruzioni, selezione livelli e della partita in corso.

- **Punti di forza:** L'uso di Optional e Stream nel MatchManager e Cell ha reso il codice molto più leggibile e resistente agli errori, eliminando i classici controlli null che rendono il codice pesante. Il salvataggio temporaneo dei match trovati da MoveEngine, accessibili tramite il metodo `getMatches()`, permette di recuperarli dopo il controllo di mossa evitando calcoli ridondanti, cosa che sprecherebbe solo tempo e risorse. Infine la centralizzazione il cambio scena nello SceneManager ha permesso di gestire il ciclo di vita delle schermate in modo pulito, evitando che le diverse View si sovrapponevano.
- **Punti di debolezza:** Il metodo `findAllMatches()` esegue una scansione completa di ogni cella della Board, includendo anche posizioni che potrebbero già far parte di un match precedentemente identificato. Sebbene il Set garantisca l'assenza di duplicati nei risultati, l'algoritmo effettua un numero di interrogazioni superiore al necessario, sarebbe quindi potuta essere scritta in modo più efficiente. Inoltre, alcuni metodi all'interno della GameView legati alla Timeline presentano una logica annidata densa; avrei potuto suddividere meglio le responsabilità in metodi ausiliari più piccoli per facilitare la leggibilità.

4.1.4 Anna Pollini

Durante lo sviluppo del progetto, mi sono occupata della modellazione e creazione degli obiettivi di gioco e della gestione logica della sessione di partita. Inoltre ho progettato ed implementato i controller dell'applicazione (AppController e GameController), i quali fungono da intermediari per la gestione delle informazioni del model.

- **Punti di forza:** Le classi Session e Goal presentano un elevato livello di disaccoppiamento rispetto al resto del dominio logico (fatta eccezione per le interazioni con la configurazione dei livelli). Grazie alle scelte di design adottate, entrambe le classi mantengono responsabilità ben definite e limitate, risultando facilmente leggibili e manutenibili. In ottica di estendibilità e riutilizzo del codice, sono stati utilizzati due design pattern: il *pattern Simple factory* per l'istanziamento dinamica dei goal e il pattern Command all'interno dell'AppController per la gestione flessibile degli eventi. Un ulteriore punto di forza è stato l'utilizzo di un Consumer<GameController> in StartLevelCommand, utilizzato come meccanismo di callback per evitare dipendenze tra comando e controller.
- **Punti di debolezza:** A causa delle numerose dipendenze del GameController verso le componenti del Model, nelle fasi iniziali si sono riscontrate alcune difficoltà nel coordinamento con gli altri componenti del gruppo. Questo ha richiesto diverse attività di refactoring rispetto all'idea progettuale originale, in modo da poter garantire una maggiore armonia ed integrazione tra tutte le componenti. Inoltre, per via delle stringenti tempistiche di consegna, non è stato possibile introdurre un'altra tipologia di goal.

4.2 Difficoltà incontrate e commenti per i docenti

La sfida principale è stata l'organizzazione del lavoro in gruppo: coordinare le attività per evitare interferenze tra le diverse componenti del sistema e gestire l'integrazione del codice scritto dai singoli membri si è rivelato complesso.

Per fortuna, grazie ad un'efficace comunicazione collettiva, siamo stati in grado di comprendere appieno lo scopo e le funzionalità del codice creato dagli altri membri del gruppo e tutto sommato siamo soddisfatti dell'applicazione ottenuta.

Relativamente agli strumenti, l'utilizzo di Git si è rivelato inizialmente ostico per parte del gruppo, ma, senza di esso, la gestione del progetto su più fronti sarebbe stata impossibile.

Appendice A

Guida utente

All'avvio l'applicazione presenterà una schermata con due tasti: cliccando "Play" si aprirà un ulteriore menù contenente i livelli, cliccando invece "How to Play" se ne aprirà uno contenente le istruzioni di gioco dove è presente un tasto per tornare al menù.

Una volta dentro al menù dei livelli basterà selezionarne uno per iniziare la partita, in alternativa cliccando sul tasto "Menu" in basso a sinistra si potrà tornare alla schermata iniziale.

Durante il corso della partita per poter scambiare due celle è sufficiente cliccare sulla prima e successivamente sulla seconda; se le due celle non sono adiacenti o non producono nessun match lo scambio non avrà luogo. Se l'utente non clicca una cella per 10 secondi, si illumineranno delle celle per suggerire un possibile match.

Per utilizzare i power-up basta selezionarne uno tramite i bottoni a destra della griglia di gioco e poi cliccare su una determinata cella. Se si volesse annullare la selezione di un power-up, sarà sufficiente ricliccarci sopra o cliccarne uno diverso.

In partita, cliccando sul tasto "Quit" si chiuderà la sessione di gioco e si tornerà al menù dei livelli.

Appendice B

Esercitazioni di laboratorio

B.0.1 marco.galli14@studio.unibo.it

- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p286991>

B.0.2 gianluca.marsella@studio.unibo.it

- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287189>
- Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290770>

B.0.3 sara.mazzoli8@studio.unibo.it

- Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286183>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p287270>

B.0.4 anna.pollini2@studio.unibo.it

- Laboratorio 07: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p284930>
- Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286031>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p286978>
- Laboratorio 12: <https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290723>

Bibliografia